

Engineering Reference Book

한눈에 보는 32개 키워드

01 Top-down Design

큰 그림부터, 그 다음 세부

05 MECE

겹치지 않게, 빠뜨리지 않게

09 Unit / Regression Testing

부분 점검 + 다시 점검

13 Event-Driven Thinking

~하면 ~한다

17 YAGNI

필요할 것 같다 ≠ 진짜 필요하다

21 Constant vs Variable

고정된 값 / 변하는 값

25 Flowchart

흐름을 도형으로 적기

29 Loop Boundary

한 칸 차이의 함정

02 Bottom-up Design

작은 부품부터, 그 다음 큰 그림

06 Scope Management

어떤 이름이 어디까지 통하는가

10 Generalization

공통 패턴 뽑아내기

14 Event Handler

사건을 받아 처리하는 절차

18 Inheritance

물려받고, 더한다

22 Type Annotation

이 칸에는 무엇이 들어가는가

26 Pseudocode

코드 직전의 우리말

30 Termination Condition

언제 멈출지부터 정한다

03 Tree

한 뿌리에서 가지로, 가지에서 잎으로

07 Interface

두 시스템이 만나는 약속

11 Boundary Condition

이 값을 넘느냐 마느냐로 갈린다

15 Production Rule

If · Then 한 줄짜리 규칙

19 Function

입력 → 처리 → 출력

23 Qualitative vs Quantitative

느낌으로? 숫자로?

27 Sequence

순서가 결과를 바꾼다

31 Control Structures

흐름의 세 가지 뼈대

04 Divide and Conquer

쪼개고, 풀고, 합친다

08 Input-Output

넣은 만큼 나온다

12 Trigger

동작을 깨우는 신호

16 Abstraction

복잡함에서 핵심만 남기기

20 Parameterization

고정 값을 '바꿀 수 있는 값'으로

24 Value Range

최솟값에서 최댓값까지

28 Iteration

조건이 풀릴 때까지 다시

32 True / False Logic

세상을 두 갈래로 좁히기

01 — DECOMPOSITION

분해

큰 문제를 다룰 만한 조각으로 자르고, 그 경계를 약속한다.

01

Top-down Design

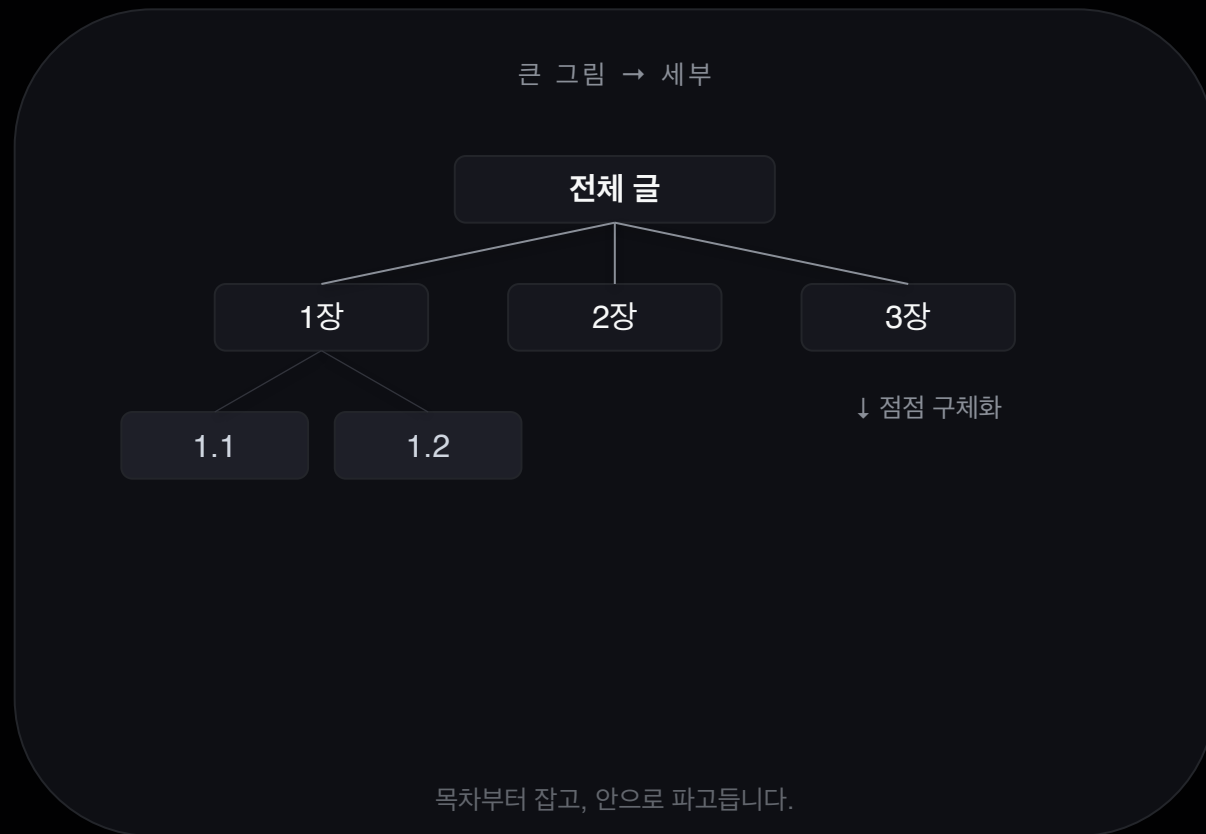
큰 그림부터, 그 다음 세부

핵심 개념

전체 구조를 먼저 잡고, 그 안을 점점 구체화한다.

글을 쓰기 전 목차를 잡듯, 시스템을 만들기 전에 전체의 뼈대를 그립니다. 세부부터 시작하면 큰 그림이 휘청입니다. 단, 작은 부품의 현실적 제약을 모르고 위에서만 그리면 실제로 만들기 어려운 설계가 나올 수 있어, 보통 Bottom-up과 번갈아가며 보완합니다.

출처 · Wirth (1971), Program Development by Stepwise Refinement.



Bottom-up Design

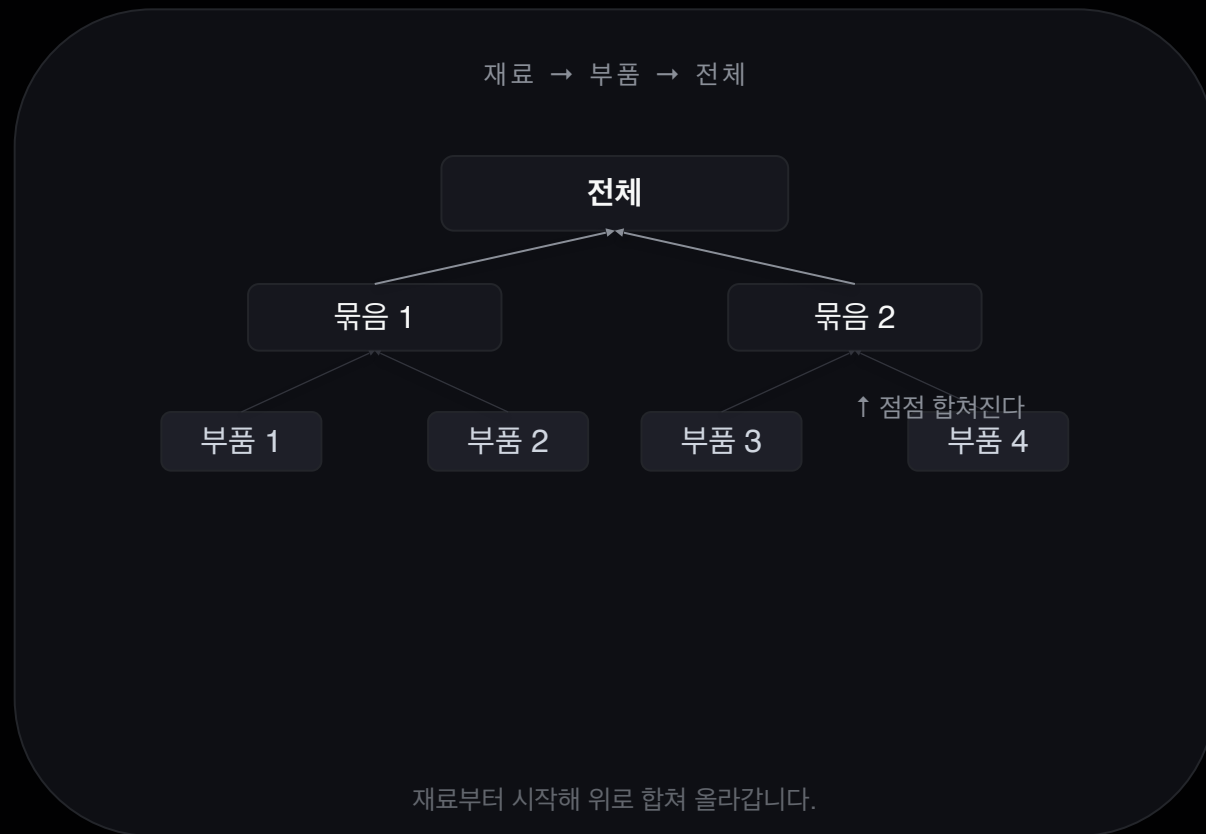
작은 부품부터, 그 다음 큰 그림

핵심 개념

잘 만들어진 작은 부품을 먼저 두고, 그것들을 모아 더 큰 구조로 쌓아 올린다.

Top-down이 ‘목차부터’라면, Bottom-up은 ‘재료부터’입니다. 신뢰할 수 있는 작은 부품을 먼저 검증해두면, 위로 합쳐지는 구조도 자연스럽게 견고해집니다. 두 방식은 서로 보완 관계라 — 큰 그림을 그리는 Top-down과, 만들 수 있는지를 검증하는 Bottom-up을 번갈아가며 진행하는 게 보통입니다.

출처 · 일반적 개념.



Tree

한 뿌리에서 가지로, 가지에서 잎으로

핵심 개념

한 부모 아래 자식이 달리고, 자식이 또 자식을 가지는 계층 구조.

폴더 구조나 회사 조직도처럼, 부모와 자식의 관계가 분명한 곳이라면 어디에나 트리가 있습니다. 위에서 아래로 한 단계씩 좁혀지며, 거꾸로 보면 ‘분해의 결과’가 그대로 드러납니다.

출처 · 일반적 개념.



Divide and Conquer

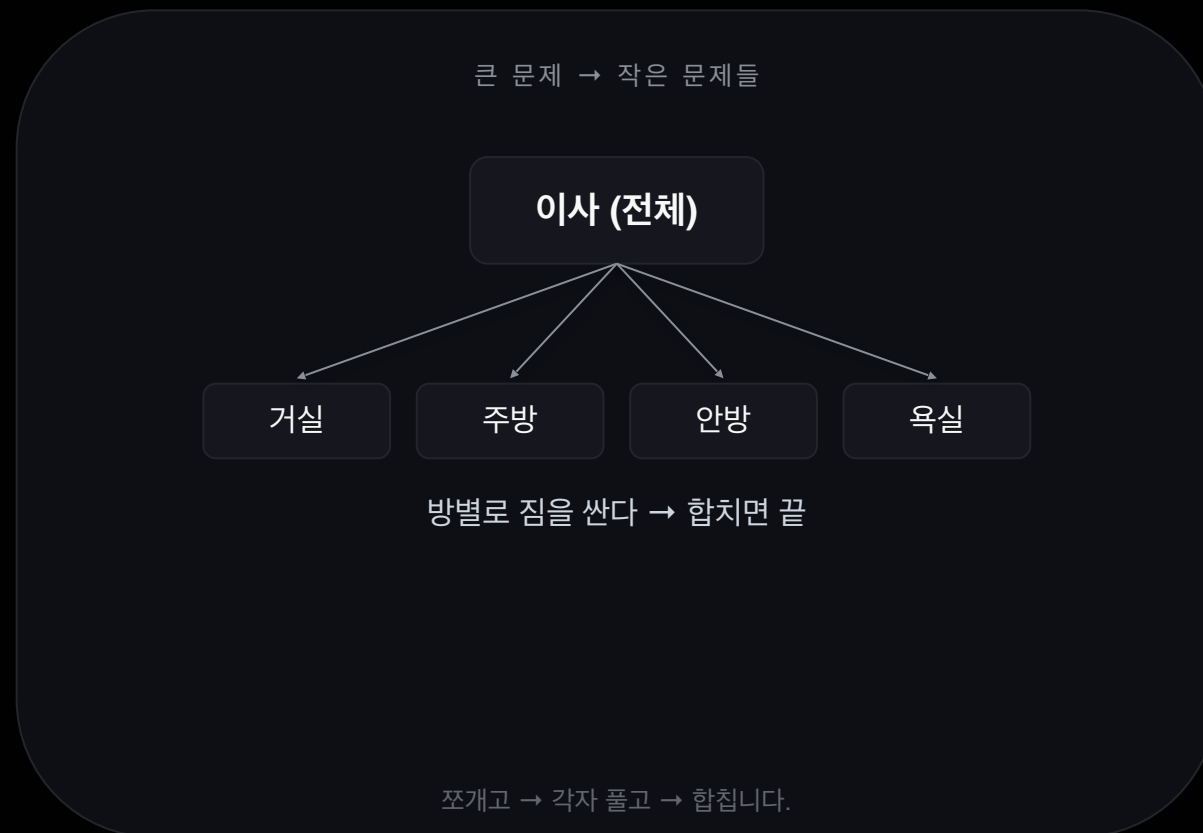
쪼개고, 풀고, 합친다

핵심 개념

큰 문제를 작은 문제 여럿으로 나눠 각각 해결한 뒤 합친다.

이사 짐 싸기를 ‘집 전체’가 아니라 ‘방별로’ 나누면 단번에 만만해집니다. 쪼개는 기준만 잘 잡으면 어떤 큰 문제도 다룰 만해집니다.

출처 · 일반적 개념.



MECE

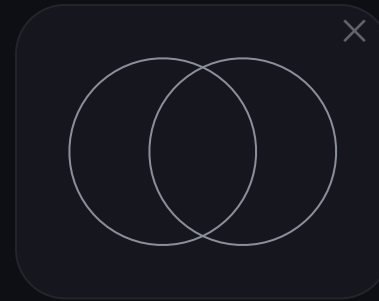
겹치지 않게, 빠뜨리지 않게

핵심 개념

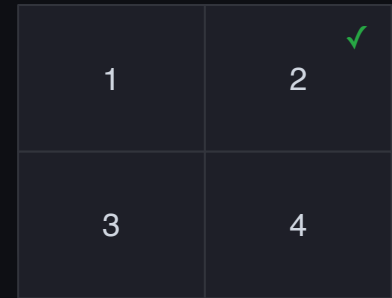
분류는 서로 겹치지 않고(Mutually Exclusive), 빠짐도 없어야
(Collectively Exhaustive) 한다.

사람을 ‘성인/미성년’으로 나누면 MECE이지만, ‘학생/직장인’으로 나누면
학생 직장인은 양쪽에 들어가고, 어린이는 어디에도 안 들어갑니다.

MUTUALLY EXCLUSIVE · COLLECTIVELY EXHAUSTIVE



겹침 + 빠짐



겹치지 않고, 빠지지 않게

분류는 겹치지 않게, 그리고 빠지지 않게.

출처 · Minto (1987), The Pyramid Principle.

Scope Management

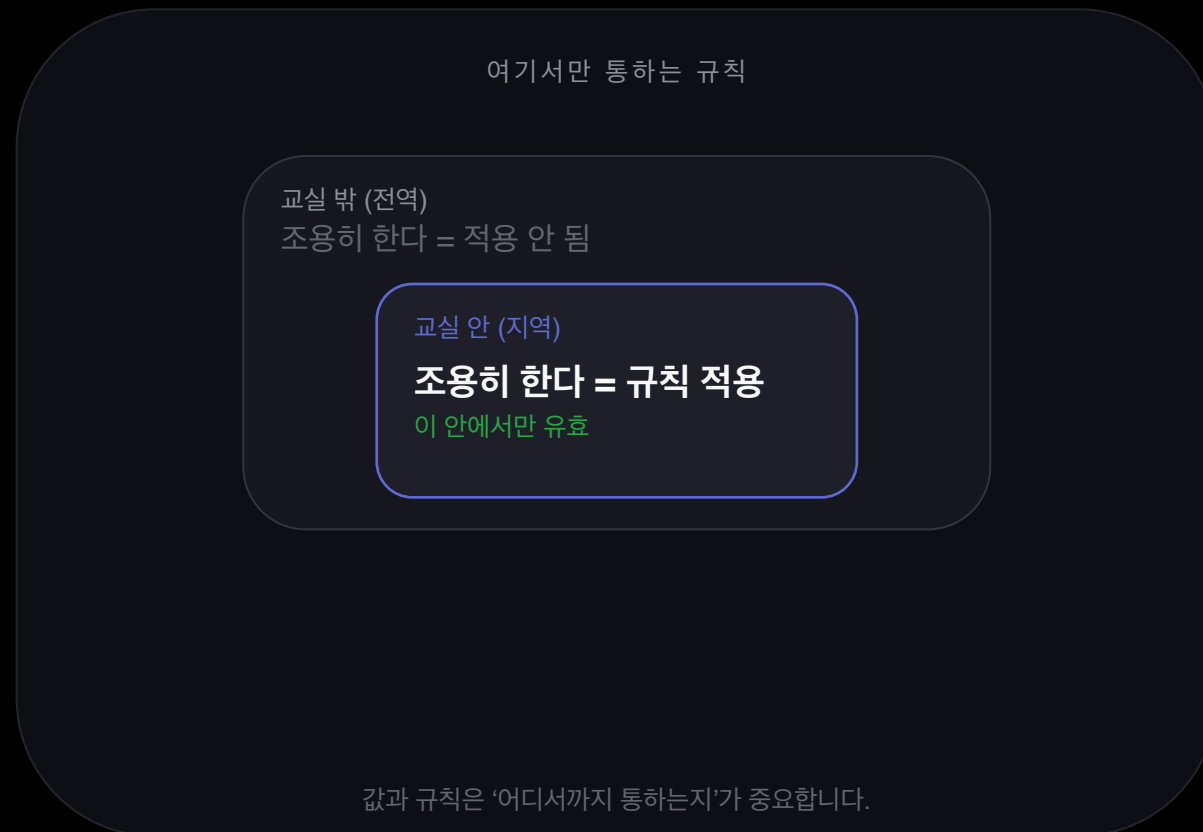
어떤 이름이 어디까지 통하는가

핵심 개념

값·규칙·이름이 유효한 범위를 명확히 그어두고 관리한다.

교실 안 규칙은 교실 밖에서 적용되지 않습니다. 변수도 마찬가지로 ‘이 안에서만 통한다’를 명시해두면, 같은 이름을 다른 곳에서 또 써도 부딪히지 않고, 안에서 일어난 변화가 바깥으로 새지 않습니다. 영역은 보통 안과 밖이 중첩된 구조라, 안쪽에서는 바깥의 약속을 빌려 쓸 수 있지만 그 반대는 막혀 있습니다. 작게 잡힌 영역일수록 영향 범위가 좁아 안전하고, 크게 잡힌 영역일수록 어디서든 쓸 수 있는 대신 충돌의 위험이 커집니다.

출처 · 일반적 개념.



값과 규칙은 ‘어디서까지 통하는지’가 중요합니다.

Interface

두 시스템이 만나는 약속

핵심 개념

서로 다른 두 시스템이 만나는 접점에 정해진 ‘모양 약속’.

콘센트 모양이 정해져 있어야 어떤 플러그든 꽂힙니다. 안쪽이 어떻게 작동하는지는 몰라도, 모양만 맞으면 연결됩니다.

출처 · Parnas (1972).



Input-Output

넣은 만큼 나온다

핵심 개념

모든 시스템은 무언가를 넣으면(입력) 무언가가 나오는(출력) 구조다.

토스터에 빵을 넣으면 토스트가 나옵니다. 어떤 시스템이든 ‘무엇을 받아서 무엇을 돌려주는가’로 끊어서 보면 단순해집니다. 입력과 출력이 명확하면, 안에서 무엇이 일어나든 바깥에서는 같은 방식으로 사용할 수 있습니다.

출처 · 일반적 개념.



02 — PATTERN RECOGNITION

패턴 인식

사례들에서 반복되는 구조를 찾아, 새로운 사례까지 같은 방식으로 다룬다.

02

Generalization

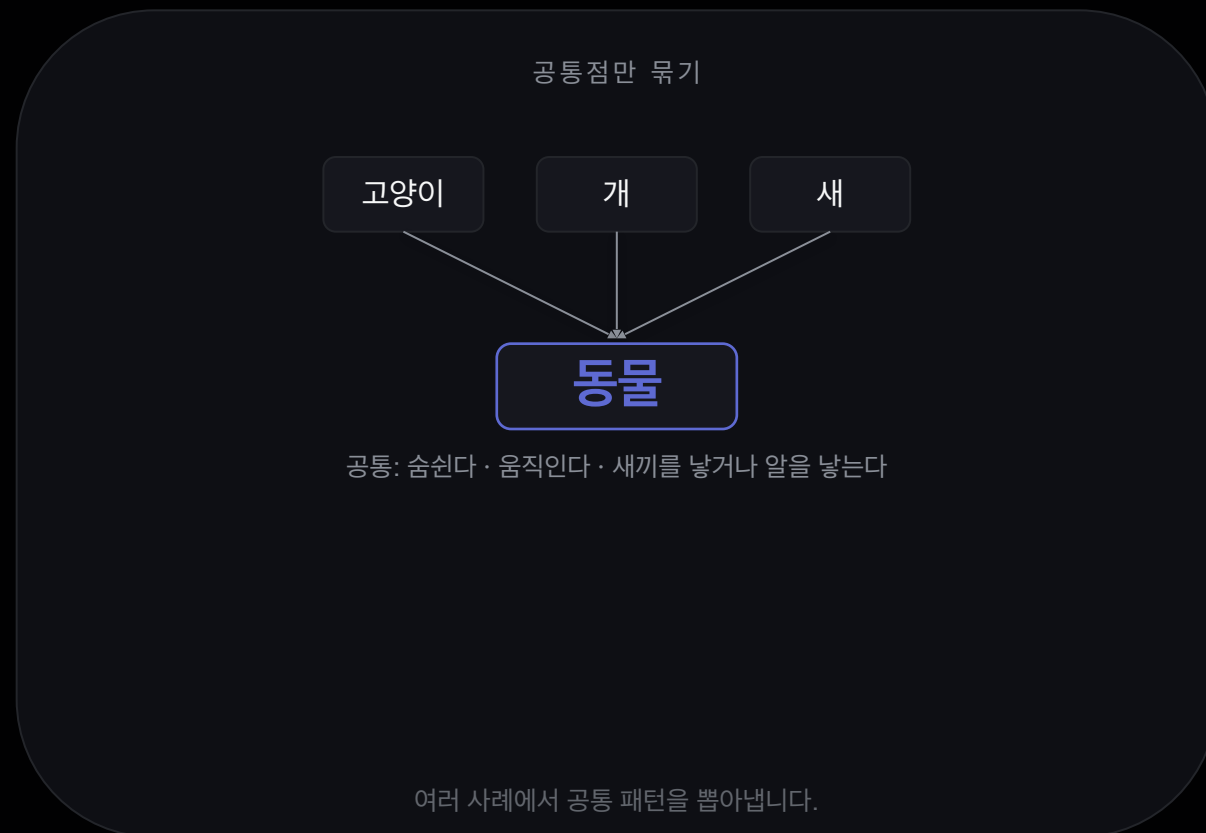
공통 패턴 뽑아내기

핵심 개념

여러 사례에서 공통점만 찾아 한 단계 위의 개념을 만든다.

고양이·개·새는 다 다르지만 ‘동물’이라는 더 큰 범주로 묶을 수 있습니다. 일반화는 처음 보는 사례까지 같은 규칙으로 다룰 수 있게 만들어줍니다.

출처 · 일반적 개념.



Boundary Condition

이 값을 넘느냐 마느냐로 갈린다

핵심 개념

어떤 기준 수치가 안과 밖을 가르고, 그 경계를 넘는 순간 상태나 동작이 달라진다.

엘리베이터 정원이 15명이라면, 15명까지는 일반 운행이고 16명부터는 안전장치가 작동합니다. 같은 시스템이 같은 입력에 대해서도 이 경계를 사이에 두고 완전히 다른 상태로 들어가는 것이죠. 그래서 시스템을 만들 때는 ‘경계가 어디인가’를 먼저 정의하고, 안쪽 동작과 바깥쪽 동작을 따로 적어둡니다.

출처 · 일반적 개념.



Trigger

동작을 깨우는 신호

핵심 개념

어떤 동작을 시작하게 만드는 조건이나 사건.

문 앞에 사람이 서면 자동문이 열립니다. ‘무엇이 깨우는가’를 명확히 적어두면 시스템이 언제 반응할지 예측할 수 있습니다.

출처 · 일반적 개념.

이 사건이 동작을 깨운다

사람이 문 앞에
TRIGGER 조건

센서

문이 열린다
결과 동작

‘무엇이 깨우는가’ — 시스템의 시동 키.

Event-Driven Thinking

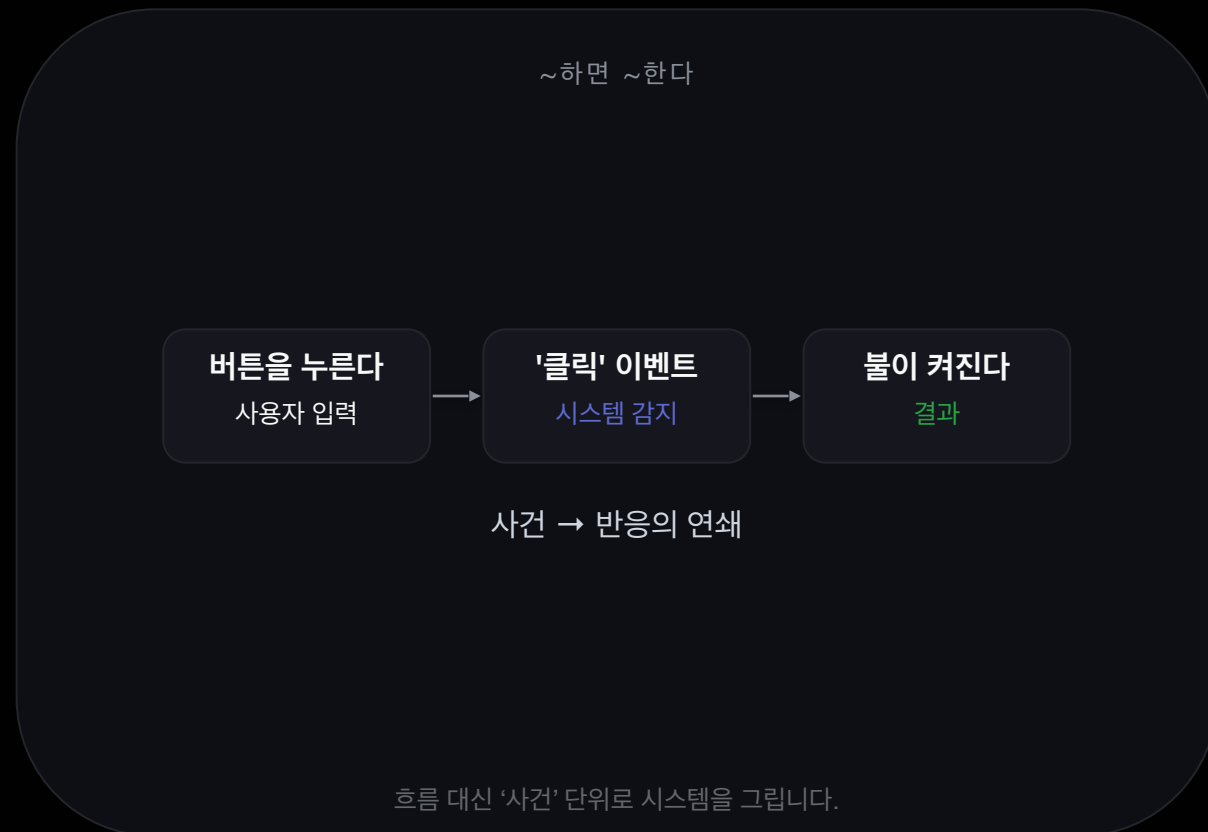
~하면 ~한다

핵심 개념

‘이런 사건이 일어나면, 이런 동작이 따라온다’로 시스템을 설계한다.

버튼을 누르면 불이 켜집니다. 흐름을 미리 다 짜두는 대신, 일어날 수 있는 사건과 그 반응을 짝지어 놓는 방식입니다.

출처 · 일반적 개념.



Event Handler

사건을 받아 처리하는 절차

핵심 개념

특정 사건이 발생했을 때 자동으로 실행되는 처리 절차.

화재 경보가 울리면 스프링클러가 작동합니다. 사건과 그 사건을 받는 처리는 늘 한 짝으로 묶어 설계합니다.

출처 · 일반적 개념.



Production Rule

If · Then 한 줄짜리 규칙

핵심 개념

‘만약 A이면 B를 한다’ 형태의 조건-행동 한 쌍.

체온이 38°C 이상이면 해열제를 복용한다. 인공지능 초기의 전문가 시스템도, 디자인 가이드라인도 결국 이 규칙들의 모음입니다.

출처 · Newell & Simon (1972).

IF · THEN

IF 체온 $\geq 38^{\circ}\text{C}$

THEN 해열제를 복용한다

“이런 상황이면 이렇게” 한 쌍의 규칙.

03 — ABSTRACTION

추상화

복잡함에서 핵심만 남기고 나머지를 숨긴다.

03

Abstraction

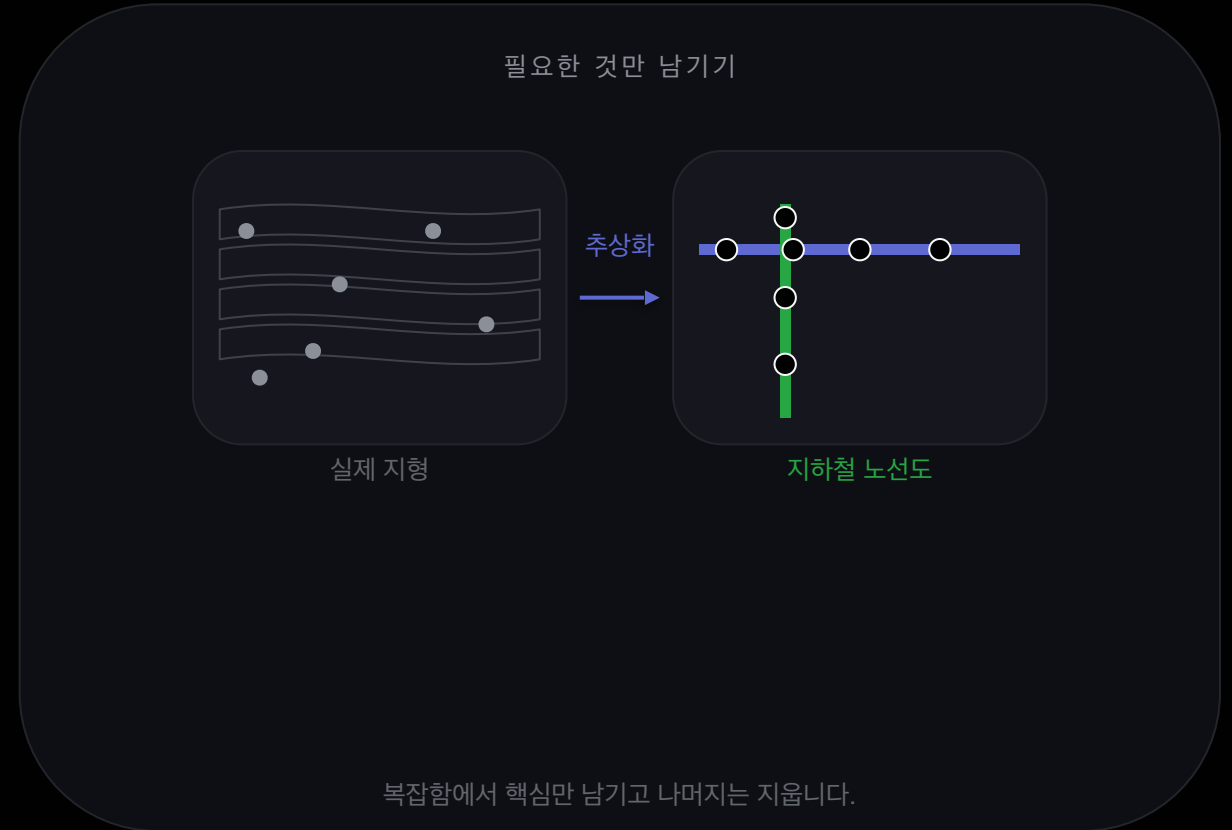
복잡함에서 핵심만 남기기

핵심 개념

복잡한 대상에서 본질만 추리고 나머지를 숨기는 것.

지하철 노선도는 실제 거리와 지형을 모두 지운 채, 역과 환승만 남긴 결과입니다. 일상에서 ‘추상적이다’가 흔히 ‘모호하다·뽀구름 같다’로 쓰이는 것과 달리, 컴퓨터과학에서 말하는 추상화는 오히려 그 반대 — 무엇을 남기고 무엇을 지울지를 명확히 결정해, 정해진 목적에 가장 적합한 단순한 모델을 만드는 일을 가리킵니다. 좋은 추상은 결국 ‘무엇을 지웠는가’가 결정합니다.

출처 · 일반적 개념.



YAGNI

필요할 것 같다 ≠ 진짜 필요하다

핵심 개념

지금 당장 필요하지 않은 것은 만들지 마라.

‘쓸 일이 있을지 모른다’는 이유로 미리 만들어둔 기능은 대개 쓰이지 않고, 그 사이 코드는 복잡해집니다. 필요할 때 만드는 것이 가장 빠르고 단순합니다.

출처 · Beck (1999), Extreme Programming Explained.

NOW VS JUST-IN-CASE

지금 필요한 것

기능 A

단순 · 명확

쓸 것 같아 미리

기능 A

기능 B? ×

기능 C? ×

기능 D? ×

복잡함만 늘어남

필요할 때 만들면 충분합니다.

Inheritance

물려받고, 더한다

핵심 개념

기존 설계의 특성을 물려받아 그 위에 새로운 특성을 더한다.

‘탈것’의 공통 특성을 ‘자동차’가 물려받고, 자동차에 엔진 종류를 더해 ‘전기차/가솔린차’로 확장합니다.

물려받기 + 확장하기



공통 특성은 위에서, 차이는 아래에서 더합니다.

출처 · Dahl & Nygaard (1967), SIMULA 67.

Function

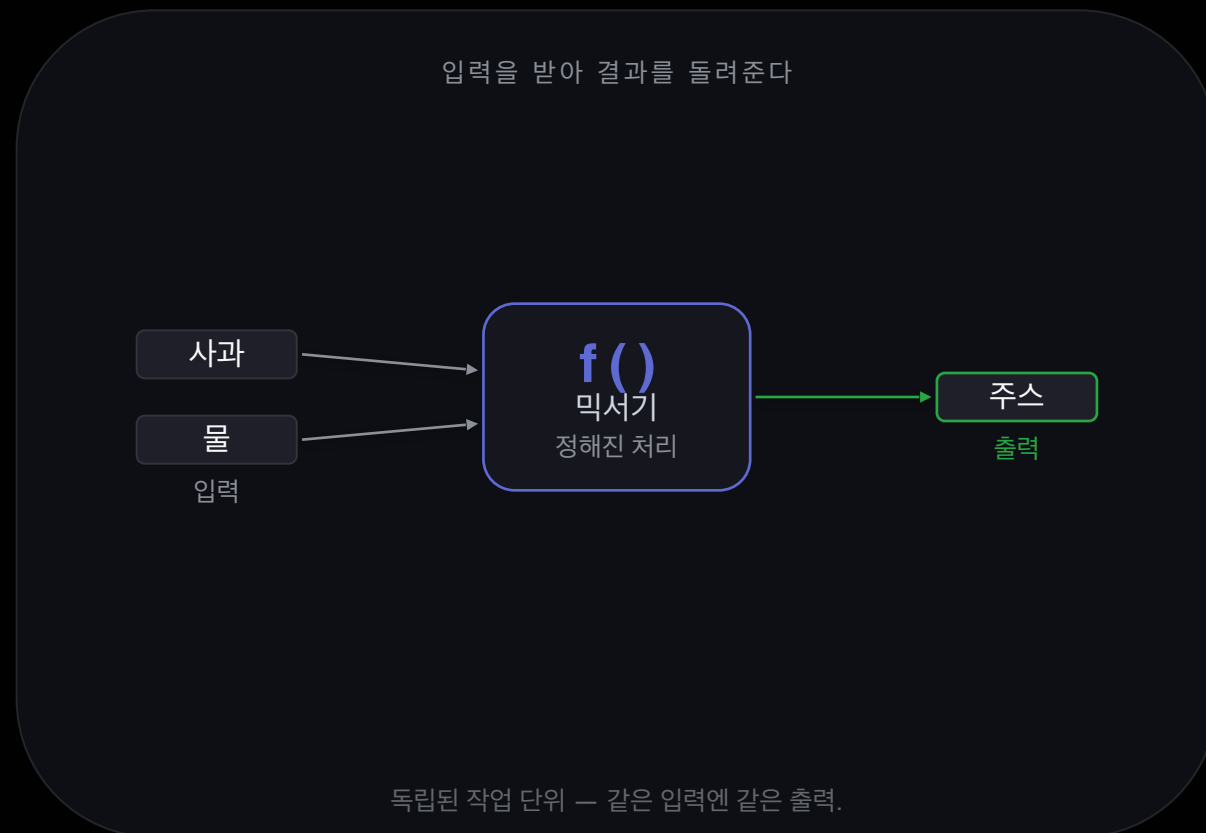
입력 → 처리 → 출력

핵심 개념

입력을 받아 정해진 처리를 한 뒤 결과를 돌려주는 독립된 작업 단위.

재료를 넣으면 주스가 나오는 믹서기처럼, 같은 입력엔 같은 출력이 나오도록 만들면 어디에나 끼워 쓸 수 있습니다.

출처 · 일반적 개념.



Parameterization

고정 값을 ‘바꿀 수 있는 값’으로

핵심 개념

고정된 값 대신 바꿀 수 있는 값을 넘겨, 같은 구조를 여러 상황에 재사용한다.

레시피의 ‘2인분’을 ‘N인분’으로 바꾸면, 같은 카드 한 장으로 어떤 손님 수에도 대응할 수 있습니다.

출처 · 일반적 개념.

값만 바꿔서 다시 쓰기

파스타 레시피

분량 **N 인분** → 1, 2, 4, 6 ...

면 100g × N
물 500ml × N

고정 값 → 바꿀 수 있는 값으로 만들면 재사용됩니다.

Constant vs Variable

고정된 값 / 변하는 값

핵심 개념

변하지 않는 값(상수)과, 상황에 따라 바뀌는 값(변수)을 구분한다.

원주율 3.14는 상수, 오늘의 기온은 변수입니다. 이 구분이 흐릿하면 ‘바뀌면 안 되는 값’이 슬그머니 바뀌는 사고가 납니다.

출처 · 일반적 개념.



Type Annotation

이 칸에는 무엇이 들어가는가

핵심 개념

값이 ‘숫자/글자/참거짓’ 중 어떤 것인지 미리 표시해두는 것.

‘나이 칸엔 숫자만’이라고 적어두면, 이상한 입력은 입구에서 막힙니다. 타입은 시스템과 사용자 사이의 가장 작은 약속입니다.

출처 · 일반적 개념.

이 칸엔 무엇이 들어가는가

이름	김모래	TEXT
나이	27	NUMBER
동의	✓	TRUE / FALSE

타입을 미리 적어두면, 이상한 입력은 입구에서 막힙니다.

Qualitative vs Quantitative

느낌으로? 숫자로?

핵심 개념

같은 현상도 ‘질적’ 표현과 ‘양적’ 표현 두 가지로 말할 수 있다.

‘따뜻하다’는 느낌·맥락에 강하고, ‘25°C’는 비교·계산에 강합니다. 어느 쪽으로 말할지를 정하는 일이 곧 데이터 설계의 시작입니다.

출처 · 일반적 개념.

FEELING VS NUMBER

질적 (QUALITATIVE)

따뜻하다

느낌 · 묘사 · 분위기
감정/맥락에 강함

양적 (QUANTITATIVE)

25°

수치 · 비교 · 계산
정확/판단에 강함

같은 현상이라도 두 언어로 말할 수 있습니다.

Value Range

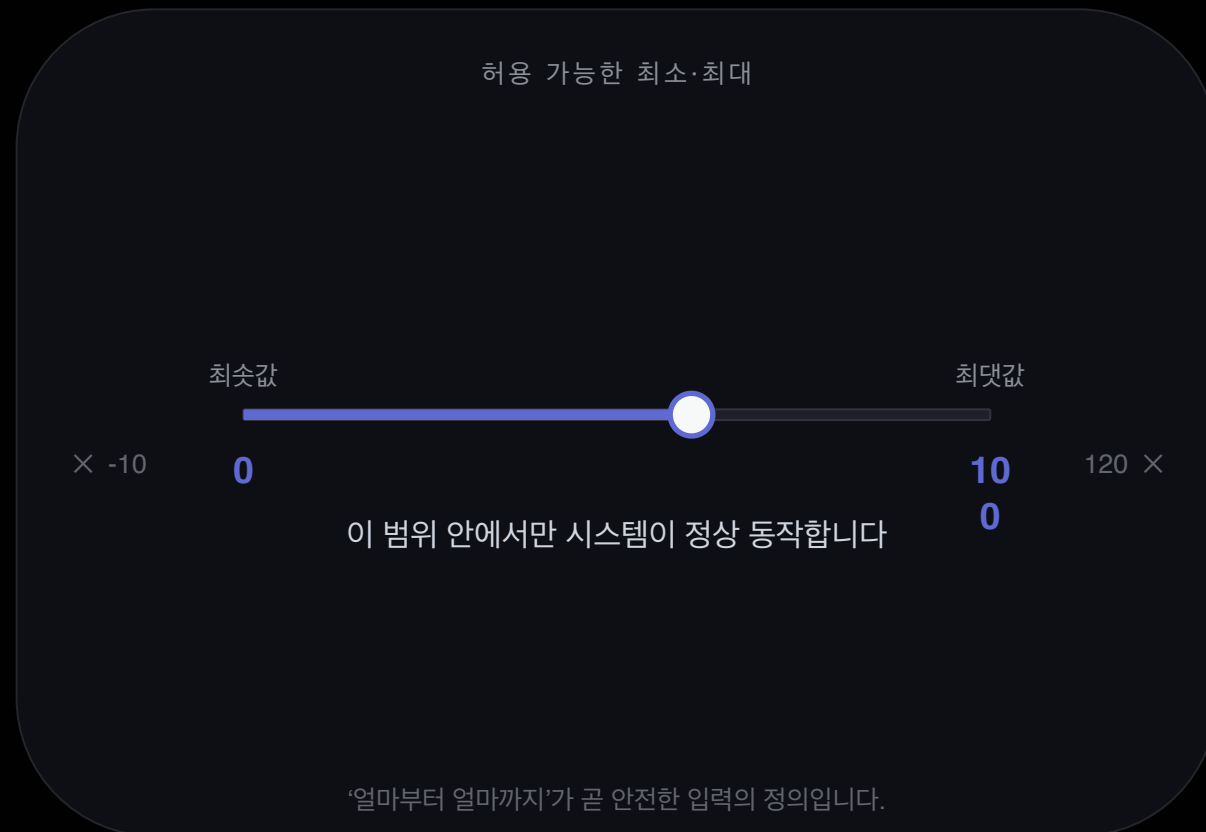
최솟값에서 최댓값까지

핵심 개념

값이 가질 수 있는 최솟값부터 최댓값까지의 범위.

볼륨은 0~100, 색상의 알파는 0~1. 시스템이 정상 동작하려면 ‘얼마부터 얼마까지’를 미리 정해두고 그 밖의 값은 막아야 합니다.

출처 · 일반적 개념.



04 — FLOWCHART

플로우차트

절차를 도형과 화살표로 적어 그대로 작동하게 만든다.

04

Flowchart

흐름을 도형으로 적기

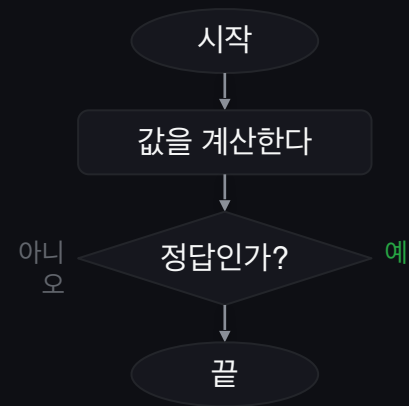
핵심 개념

판단과 행동의 흐름을 도형과 화살표로 그린 그림.

원은 시작/끝, 사각은 처리, 마름모는 판단입니다. 머릿속의 절차를 종이 위에 옮겨두면 빈틈이 보입니다.

출처 · Goldstine & von Neumann (1947).

표준 도형으로 그리기



원=시작/끝, 사각=처리, 마름모=판단.

Pseudocode

코드 직전의 우리말

핵심 개념

프로그래밍 언어가 아닌 일상어로 로직을 적는 것.

‘만약 비가 오면 우산 챙기기, 아니면 그냥 나가기’처럼, 들여쓰기와 조건어를 빌려 적습니다. 어떤 언어로 옮기든 거의 그대로 따라갑니다.

출처 · 일반적 개념.

프로그래밍 언어가 아닌 우리말로

PSEUDOCODE.TXT

만약 비가 온다면
 우산을 챙긴다
아니라면
 그냥 나간다

그림보다 정확하고, 코드보다 부담 없이.

Sequence

순서가 결과를 바꾼다

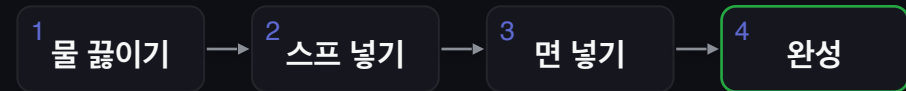
핵심 개념

일이 일어나는 순서. 같은 재료라도 차례가 달라지면 다른 결과가 나온다.

라면을 끓일 때 물을 끓이기 전에 면을 넣으면 같은 라면이 안 됩니다. 흐름을 짤 때 가장 먼저 점검할 것은 ‘이 순서가 맞는가’입니다.

출처 · 일반적 개념.

순서가 결과를 바꾼다



스프를 면 다음에 넣으면 같은 라면이 아닙니다.

Iteration

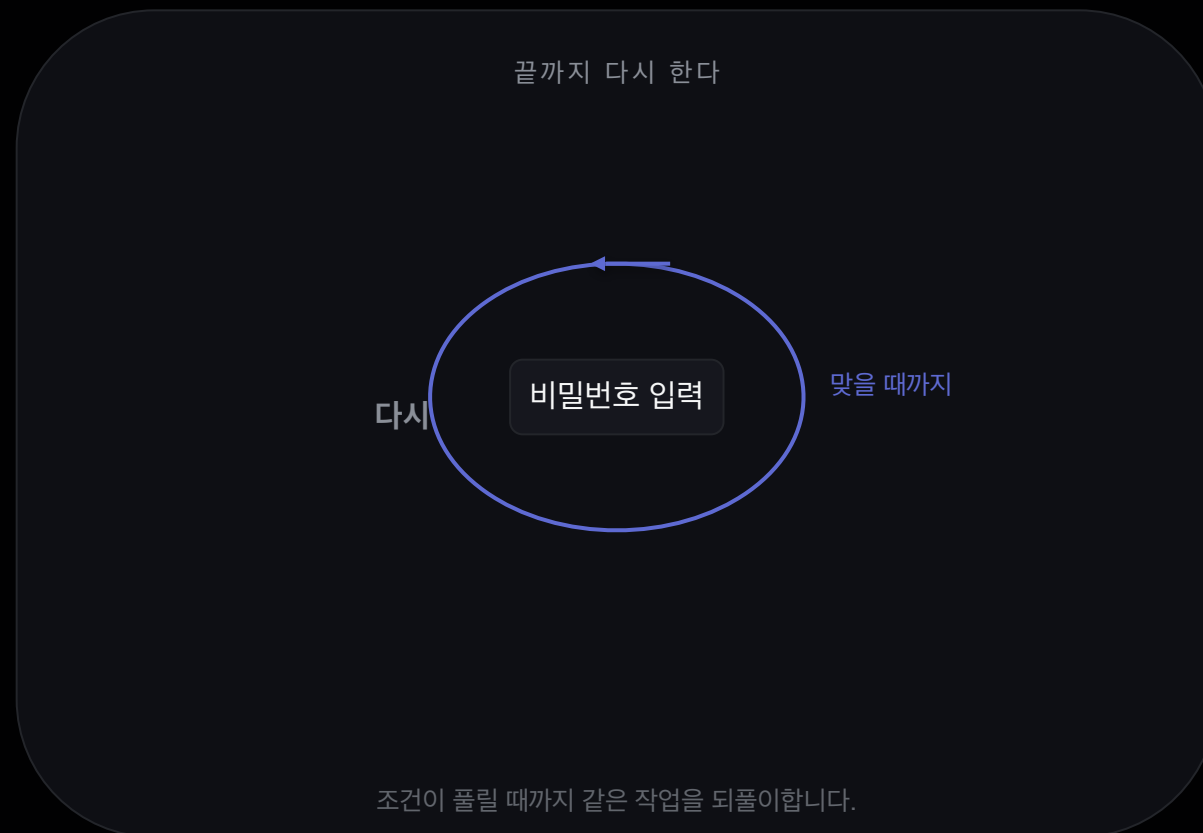
조건이 풀릴 때까지 다시

핵심 개념

같은 작업을 조건이 충족될 때까지 반복하는 것.

비밀번호가 맞을 때까지 다시 묻고, 페이지가 끝까지 보일 때까지 스크롤이 이어집니다. ‘언제 끝나는가’를 함께 정해야 무한 루프를 피합니다.

출처 · 일반적 개념.



Loop Boundary

한 칸 차이의 함정

핵심 개념

반복이 '어디서 시작하고 어디서 끝나는지'를 정확히 정한다.

'1번부터 10번까지'가 1을 포함하는지, 10을 포함하는지에 따라 결과가 한 항목씩 달라집니다. 경계 한 칸이 가장 자주 일으키는 버그입니다.

출처 · 일반적 개념.

시작 · 끝을 찍는다

시작

끝



1



10

1번이 포함? 10번이 포함?

한 칸 차이로 결과가 바뀝니다.

Termination Condition

언제 멈출지부터 정한다

핵심 개념

반복이나 프로세스를 멈추는 명확한 기준.

물이 끓으면 불을 끈다 — 끝나는 신호가 있어야 동작이 끝납니다. 종료 조건이 없는 자동화는 영원히 도는 위험한 시스템입니다.

출처 · 일반적 개념.



Control Structures

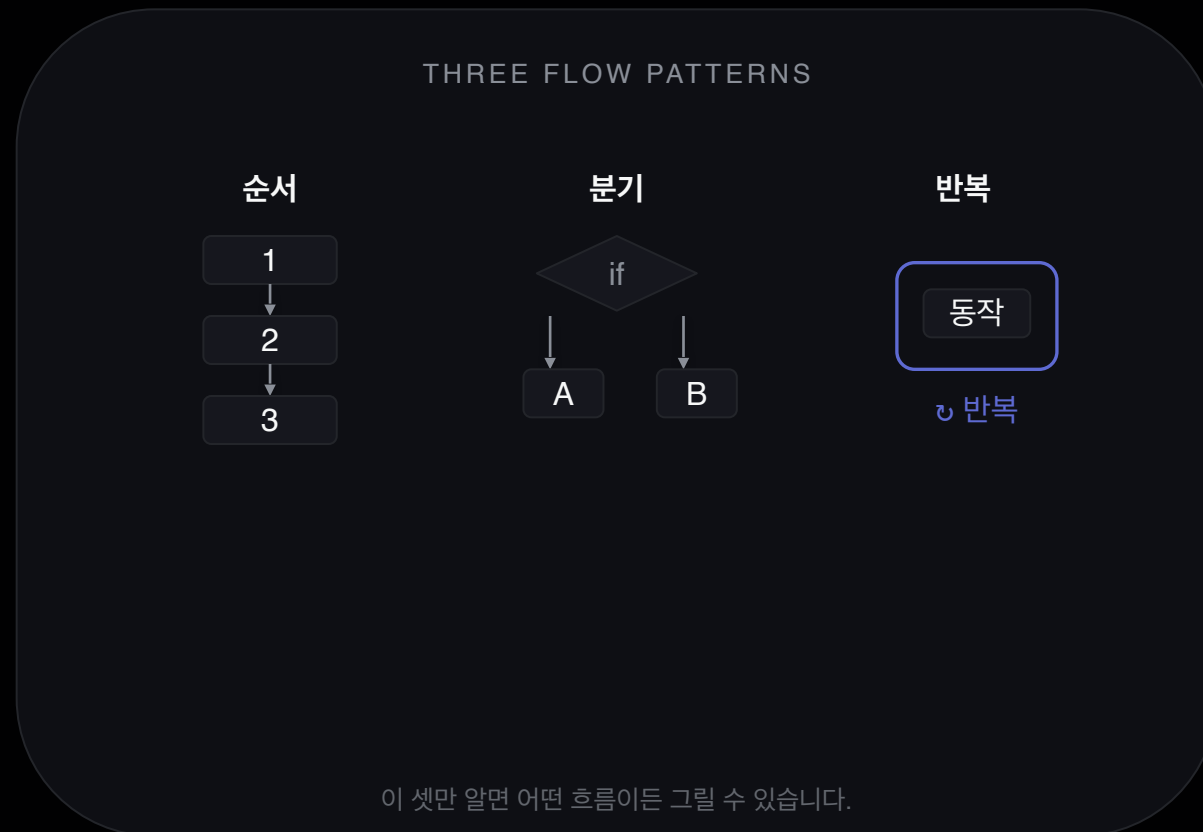
흐름의 세 가지 뼈대

핵심 개념

어떤 프로그램이든 결국 순서·분기·반복의 조합으로 표현된다.

1966년 뵘과 야코피니가 증명했듯, 이 셋만으로 모든 알고리즘을 적을 수 있습니다. 새로운 흐름을 그릴 때도 이 셋부터 떠올리세요.

출처 · Böhm & Jacopini (1966).



True / False Logic

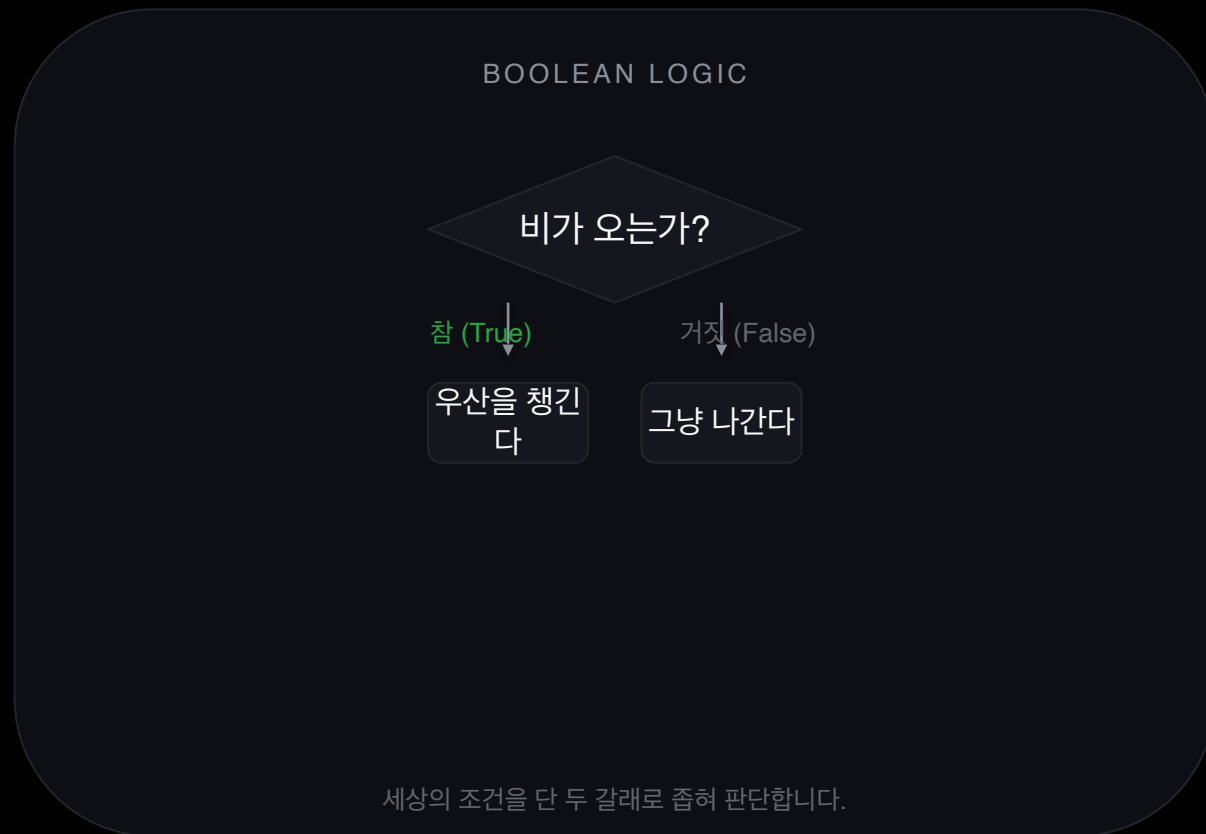
세상을 두 갈래로 좁히기

핵심 개념

조건을 ‘맞다 / 아니다’ 두 갈래로 좁혀서 다음 동작을 결정한다.

‘비가 온다 = 참’이면 우산을 챙기고, ‘거짓’이면 그냥 나갑니다. 모든 분기는 결국 이 두 값으로 환원됩니다.

출처 · Boole (1854), An Investigation of the Laws of Thought.



변경 이력

V1 2026.05.08
— 첫 발행.

CURRENT